SIT210 – EMBEDDED SYSTEM

Task 1.1

Project Description

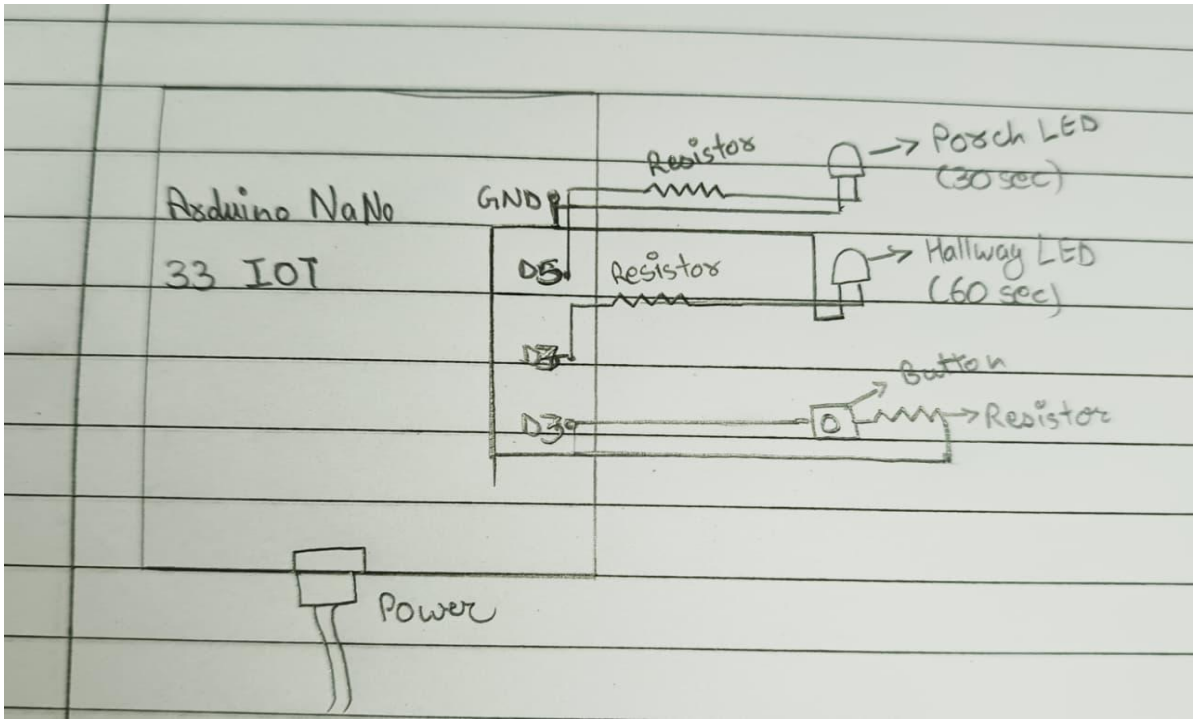
In this project we built a simple home lighting system using Arduino nano 33 IOT which uses one push button to control two LEDs that are as porch light and a hallway light. When the button is pressed both lights turn on. The porch light stays on for 30 seconds and then turns off, but the hallway light remains on for 30 seconds more before turning off automatically. This explains a basic automatic lighting system that could be used in a house.

Code Overview

So, the code starts by assigning pins for the porch LED, hallway LED, and push button. In the setup() function, the LEDs are configured as outputs and the button is configured as an input using the internal pull-up resistor. Both LEDs are initially off. The loop() here functions by continuously checking whether the button has been pressed or not. When the button is pressed, both LEDs turn on at the same time. After 30 seconds, the porch LED turns off. The hallway LED stays on for an additional 30 seconds and then turns off automatically.

At last, the program waits until the button is released before allowing the system to run again for very short time. This prevents the button press from making the lighting sequence occur multiple times while it is still being pressed down.

video link: https://youtu.be/IM2In8XgSzE



Discussion on Modular Programming

In this project, modular programming helped me making the code easier to understand and manage it. My program is divided into different parts, such as the setup() function for initializing the LEDs and button, and the loop() function for continuously checking the button state and controlling the lighting sequence.

This approach made program more organized because each part of the code has specific purpose. If new features need to be added later in code, such as additional lights or sensors, they can be added without changing the entire program. Modular programming also makes debugging easier because problems can be identified within a specific section of the code rather than searching through the whole program.

Design Perspective: Problems with the Solution

Although here the solution works correctly, there are some design limitations.

1. The program uses delay() functions, which stop the Arduino from performing other tasks while waiting. During this time, the system cannot respond to new button presses or other sensors.
2. The lighting times are fixed and cannot be changed without edit the code.
3. The system only supports one button and two lights, makes it difficult to scale for larger smart home applications.
4. There is no feedback system to indicate when the lighting sequence is active or completed.

Modular Programs for a Smart Assisted Living System

1. Automatic Door Monitoring Module

This module could use a magnetic door sensor to detect when a door is opened or closed. It would improve home security and help caregivers monitor activity within the house.

2. Motion Detection Lighting Module

This module could automatically turn lights on when movement is detected and turn them off when no motion is present. It would help elderly users move safely around the house while reducing energy consumption.

3. Temperature and Humidity Monitoring Module

This module could continuously monitor room conditions using environmental sensors. If the temperature becomes too high or too low, the system could send alerts or activate cooling and heating devices to maintain comfort and safety.

Using separate modules for these functions would make the overall smart assisted living system easier to develop, maintain, and expand in the future.